

Raport z przeglądu kodu (code review)

Strona portfolio — Michał Stężyły · Next.js 15 + Keystatic CMS + Resend (e-mail) · hosting: Vercel

Data: 11 czerwca 2026 · Przegląd wykonany przez zespół 3 wirtualnych recenzentów: **Bezpieczeństwo**, **Wydajność**, **Niezawodność**

Dla kogo jest ten raport. Jesteś projektantem, nie programistą — dlatego cały dokument jest napisany prostym językiem. Każdy problem opisuję najpierw „po ludzku” (*co Ci realnie grozi*), a dopiero potem technicznie, ze wskazaniem dokładnego pliku i numeru linii. Terminy techniczne tłumaczę na bieżąco, a na końcu jest słowniczek.

Jak czytać oceny ważności (severity)

Etykieta	Co znaczy po ludzku
CRITICAL	Pali się — naprawić natychmiast, grozi wyciekiem danych lub całkowitą awarią.
HIGH	Poważne — naprawić w pierwszej kolejności, realnie szkodzi biznesowi lub bezpieczeństwu.
MEDIUM	Średnie — warto naprawić wkrótce, ale strona działa.
LOW	Drobne — do backlogu, kosmetyka lub mało prawdopodobny scenariusz.

Najważniejsze w jednym zdaniu: nie znaleziono żadnego problemu klasy CRITICAL (nic się „nie pali”), ale są **3 sprawy do pilnej naprawy** — z czego najgroźniejsza to taka, że **klient może wysłać wiadomość przez formularz, zobaczyć „Dziękuję”, a mail nigdy do Ciebie nie dotrze.**

1. Recenzent: Bezpieczeństwo

Szukałem: włamań, podszywania się, wycieków danych, otwartych „drzwi” w aplikacji, sekretów w kodzie, braku sprawdzania uprawnień.

HIGH S1 — Zabezpieczenia przeglądarki ustawione w złym miejscu (nie działają na żywej stronie)

Obszar: nagłówki bezpieczeństwa / konfiguracja hostingu

Opis: Wszystkie „nagłówki bezpieczeństwa” (zasady, które mówią przeglądarce m.in. „nie pozwól osadzić tej strony w cudzej ramce” i „blokuj podejrzane skrypty” — tzw. CSP, X-Frame-Options) są zapisane w pliku `netlify.toml`. Ale ten plik czyta wyłącznie hosting Netlify. Twoja żywa strona stoi na **Vercelu** (potwierdza to `.vercel/repo.json` i adres `michał-stezały.vercel.app`), a Vercel **kompletnie ignoruje** ten plik. W projekcie nie ma `vercel.json` ani sekcji `headers()` w `next.config.ts`.

Dlaczego to ryzyko: Na żywej stronie te zabezpieczenia w praktyce **nie istnieją**. Otwiera to drogę m.in. do „clickjackingu” (ktoś osadza Twoją stronę w niewidzialnej ramce na swojej i podstępem zbiera kliknięcia) oraz osłabia ochronę przed wstrzykiwaniem obcych skryptów.

Po ludzku: myślisz, że masz w mieszkaniu alarm, ale instrukcja montażu leży w szufladzie — alarm nigdy nie został podłączony.

Gdzie szukać: `netlify.toml:18–28` (tu są nagłówki) · `next.config.ts:1–35` (brak `headers()`)

Rekomendacja: Przenieść nagłówki do miejsca, które czyta Vercel — dodać funkcję `async headers()` w `next.config.ts` z tą samą listą (CSP, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy, Permissions-Policy). Wtedy zadziałają niezależnie od hostingu.

Możliwy efekt uboczny: Zbyt restrykcyjne CSP potrafi zablokować własne skrypty (np. Cloudflare Turnstile, analytics). Trzeba przekleić dokładnie te same dozwolone adresy, które są już w `netlify.toml`, i przetestować formularz po wdrożeniu.

HIGH S2 — Otwarty endpoint do kompresji obrazów (każdy z internetu może go używać)

Obszar: brak uwierzytelnienia / autoryzacji

Opis: Adres `/api/admin/compress-image` przyjmuje pliki do 25 MB i przerabia je biblioteką `sharp` (operacja obciążająca procesor i pamięć). Mimo „admin” w nazwie **nie sprawdza, kto wysyła** — nie ma logowania, hasła ani limitu liczby zapytań. Każdy w internecie może go wywołać dowolnie wiele razy.

Dlaczego to ryzyko: Ktoś może w pętli wysyłać duże pliki i (a) zapłacisz za zużyty czas serwera na Vercelu, (b) doprowadzić do przeciążenia (strona zwalnia lub przestaje odpowiadać). To klasyczny „atak na portfel” i odmowa usługi (DoS).

Po ludzku: to jak ksero w Twoim biurze wystawione na ulicę bez nadzoru — każdy może przyjść, robić tysiące kopii, a rachunek za prąd i toner przychodzi do Ciebie.

Gdzie szukać: `src/app/api/admin/compress-image/route.ts:28–67` (cała funkcja POST, brak jakiegokolwiek sprawdzenia tożsamości)

Rekomendacja: Skoro to narzędzie wewnętrzne (tak jak `save-hero`, które jest wyłączone na produkcji), najprościej: zablokować je na produkcji tak samo (`404` gdy nie development) *albo* wymagać prostego sekretu/logowania. Niezależnie — dołożyć limit zapytań (jak w formularzu).

Możliwy efekt uboczny: Jeśli korzystasz z tego narzędzia (`/admin/compress`) na żywej stronie, a nie tylko lokalnie — po zablokowaniu przestanie działać online. Wtedy lepsza opcja to logowanie/sekret zamiast pełnej blokady.

MEDIUM S3 — Limit wiadomości można obejść, podrabiając swój adres

Obszar: rate limiting / zaufanie do danych z przeglądarki

Opis: Formularz ogranicza liczbę zgłoszeń „3 na 10 minut z jednego adresu IP”. Adres odczytywany jest z nagłówka `x-forwarded-for`, biorąc jego **pierwszą** wartość — a tę wartość nadawca może dowolnie zmyślić.

Dlaczego to ryzyko: Spamer przy każdym wysłaniu wpisuje inny zmyślony adres i limit go nie dotyczy — może zasypać Twoją skrzynkę.

Po ludzku: ochroniarz wpuszcza „3 osoby o tym samym nazwisku”, ale wierzy w nazwisko podane ustnie — wystarczy za każdym razem powiedzieć inne.

Gdzie szukać: `src/app/api/kontakt/route.ts:13–21` (funkcja `getClientIp`, `xff.split(",")[0]`)

Rekomendacja: Na Vercelu używać zaufanego adresu dostarczanego przez platformę (np. nagłówek `x-vercel-forwarded-for` / pole, które ustawia sam Vercel), a nie pierwszej wartości `x-forwarded-for`. To wartość, której użytkownik nie kontroluje.

Możliwy efekt uboczny: Jeśli kiedyś zmienisz hosting, trzeba dopasować nazwę nagłówka. Ryzyko częściowo łagodzi Turnstile (patrz S4) — o ile jest włączony.

MEDIUM**S4 — Ochrona antyspamowa (Turnstile) wyłącza się sama, gdy brak klucza**

Obszar: walidacja / „fail-open” (przy braku konfiguracji przepuszcza wszystko)

Opis: Funkcja sprawdzająca „czy to człowiek, czy bot” zwraca „OK, przepuść”, jeśli na serwerze nie ustawiono klucza `TURNSTILE_SECRET_KEY`. Podobnie pole-pułapka i widżet nie pojawiają się, gdy brak klucza publicznego.

Dlaczego to ryzyko: Jeśli klucz nie zostanie poprawnie ustawiony na Vercelu (łatwa pomyłka przy wdrożeniu), antyspam **w ogóle nie działa**, a Ty o tym nie wiesz. W połączeniu z S3 ochrona formularza może spaść do zera → fala spamu.

Po ludzku: bramka antyspamowa, która przy braku konfiguracji nie krzyczy „brak ochrony!”, tylko cicho otwiera drzwi na oścież.

Gdzie szukać: `src/app/api/kontakt/route.ts:28–29 ( if (!secret) return true; )` ·
`src/components/contact-form.tsx:16, 282–288`

Rekomendacja: Na produkcji traktować brak klucza jako błąd konfiguracji (zalogować ostrzeżenie i ewentualnie odmówić wysyłki), zamiast po cichu przepuszczać. W trybie dev/preview obecne zachowanie jest OK.

Możliwy efekt uboczny: Jeśli zrobimy „brak klucza = bloku”, to przy pomyłce w konfiguracji formularz przestanie wysyłać. Bezpieczniejszy kompromis: przepuszczać tylko poza produkcją, a na produkcji głośno alarmować.

LOW**S5 — Strona „design-system” jest publicznie dostępna**

Obszar: ekspozycja wewnętrznych narzędzi

Opis: `/design-system` (paleta, komponenty) jest tylko oznaczona jako „nie indeksuj” w `robots.txt` i metadanych — ale wpisując adres ręcznie, każdy ją stworzy. To samo dotyczy `/admin/hero-editor` i `/admin/compress` (UI).

Dlaczego to ryzyko: Nie ma tu danych wrażliwych, więc ryzyko małe — głównie „nieprofesjonalnie”, że narzędzia wewnętrzne są na widoku. (Faktyczny zapis blokuje GitHub/produkcja — patrz S2 i dev-only `save-hero`.)

Po ludzku: zaplecze sklepu jest niezamknięte, ale stoją tam tylko próbki i miarka — nic cennego, ale lepiej zamknąć.

Gdzie szukać: `src/app/design-system/page.tsx:18–21` · `public/robots.txt`

Rekomendacja: Niski priorytet. Jeśli zależy: ukryć te trasy za prostym logowaniem albo wyłączyć na produkcji.

Możliwy efekt uboczny: Utrudniony własny dostęp do podglądu na żywej stronie.

2. Recenzent: Wydajność ⚡

Szukałem: wolnych miejsc, robót wykonywanych wielokrotnie bez potrzeby, dużych plików, rzeczy które „nie udźwigną” większego ruchu.

Dobra wiadomość na wstępie: strona jest w dużej części „statyczna” (generowana raz, przy publikacji), więc wiele potencjalnie wolnych operacji wykonuje się *podczas budowania*, a nie przy każdym wejściu gościa. Dlatego poniższe sprawy to głównie kwestia **skalowalności i kosztów**, a nie odczuwalnej dziś wolności.

MEDIUM P1 — Lista projektów wczytywana z dysku wielokrotnie, bez zapamiętywania

Obszar: odczyt plików / brak cache

Opis: Funkcja `getAllProjects()` przy każdym wywołaniu czyta i parsuje **wszystkie** pliki projektów z dysku. Inne funkcje (sąsiedzi projektu, grupowanie po kategorii, liczniki) wołają ją osobno — więc na jednej stronie ten sam katalog bywa skanowany kilka razy. Endpoint `/api/projekty-kolejnosc` jest dodatkowo „force-dynamic” (liczony przy każdym żądaniu).

Dlaczego to ryzyko: Przy małej liczbie projektów to niezauważalne. Ale gdy projektów przybędzie albo wzrośnie ruch na trasach dynamicznych, te same pliki czytane są w kółko — niepotrzebna praca, wolniejsze odpowiedzi, wyższe koszty.

Po ludzku: za każdym razem, gdy ktoś pyta o jeden projekt, pracownik na nowo przegląda całą szafę z dokumentami — zamiast raz spisać listę i z niej korzystać.

Gdzie szukać: `src/lib/content.ts:180–187` (`getAllProjects`) i wołające ją `:202–236` · `src/app/api/projekty-kolejnosc/route.ts:4–7`

Rekomendacja: Zapamiętać wynik raz na proces (np. funkcja `cache()` z React/Next albo prosta zmienna-pamięć), żeby pliki czytać raz, a nie przy każdym wywołaniu.

Możliwy efekt uboczny: Po zmianie treści w CMS trzeba przebudować/odświeżyć cache (na Vercelu dzieje się to przy nowym wdrożeniu — więc bezpieczne). Dla narzędzia podglądu kolejności `/api/projekty-kolejnosc` cache trzeba dobrać tak, by pokazywało aktualny stan.

MEDIUM**P2 — Kompresja obrazu trzyma cały plik (do 25 MB) w pamięci, bez limitu równoczesnych zadań**

Obszar: zużycie pamięci / brak kolejkowania

Opis: Endpoint kompresji wczytuje cały plik do pamięci i uruchamia `sharp`. Nie ma ograniczenia, ile takich operacji może iść naraz.

Dlaczego to ryzyko: Kilka dużych plików jednocześnie potrafi „zjeść” pamięć funkcji i ją wyrzucić. W połączeniu z S2 (endpoint otwarty dla każdego) to wprost zaproszenie do przeciążenia.

Po ludzku: jedna osoba przy stole może rozłożyć wielki projekt, ale gdy dziesięć osób naraz rozkłada wielkie plansze, stół się załamuje.

Gdzie szukać: `src/app/api/admin/compress-image/route.ts:53–61`

Rekomendacja: Najpierw zamknąć dostęp (S2). Limit 25 MB jest rozsądny; rozważyć niższy. To narzędzie wewnętrzne — nie wymaga skalowania.

Możliwy efekt uboczny: Obniżenie limitu odrzuci bardzo duże oryginały — ale takie i tak nie powinny iść na stronę.

LOW**P3 — Pamięć limitera zapytań rośnie i nigdy nie jest sprządana**

Obszar: wyciek pamięci (drobny)

Opis: Limiter trzyma w pamięci wpis dla każdego adresu IP, który kiedykolwiek wysłał formularz. Stare wpisy są filtrowane, ale nigdy usuwane z mapy — lista kluczy tylko rośnie przez całe życie „ciepłej” instancji.

Dlaczego to ryzyko: Bardzo powoli rosnące zużycie pamięci. Przy małym ruchu nieistotne; przy dużym i długo żyjącej instancji — może z czasem przeszkadzać.

Po ludzku: notes, w którym dopisujesz każdego gościa i nigdy nie wrywasz starych kartek — kiedyś się zapelni.

Gdzie szukać: `src/lib/rate-limit.ts:11, 18–28`

Rekomendacja: Gdy po odfiltrowaniu lista dla klucza jest pusta — usuwać klucz z mapy. (I patrz decyzja architektoniczna o wspólnym liczniku — R3.)

Możliwy efekt uboczny: Brak — to czysta poprawka porządkowa.

3. Recenzent: Niezawodność 🚑

Szukałem: braku obsługi błędów, sytuacji „dane gdzieś znikają”, braku ponawiania prób, słabego wglądu w to, że coś się psuje.

HIGH R1 — Formularz mówi „Dziękuję”, nawet jeśli mail do Ciebie nie poszedł

Obszar: ciche gubienie danych / błędna informacja zwrotna

Opis: Jeśli na serwerze brakuje klucza `RESEND_API_KEY` (usługa do wysyłki maili), serwer **nie wysyła maila**, ale zwraca odpowiedź „sukces” (`ok: true, delivered: false`). Przeglądarka sprawdza tylko, czy odpowiedź jest „udana”, i pokazuje gościowi ekran „Dziękuję za wiadomość!”. Pole `delivered: false` jest ignorowane.

Dlaczego to ryzyko: To najgroźniejszy scenariusz biznesowy w całym raporcie. Potencjalny klient pisze do Ciebie, widzi podziękowanie i czeka na odpowiedź — a Ty **nie dostajesz tej wiadomości** i nawet nie wiesz, że istniała. Tracisz zlecenie i nie masz pojęcia, że je straciłeś.

Po ludzku: klient wrzuca list do skrzynki, na której świeci napis „Dziękujemy, dostarczymy!”, ale skrzynka nie ma dna — list spada w pustkę, a obie strony myślą, że dotarł.

Gdzie szukać: `src/app/api/kontakt/route.ts:110–116` (zwraca `ok: true` mimo braku wysyłki) · `src/components/contact-form.tsx:92 ( if ( res.ok ) → pokazuje sukces)`

Rekomendacja: (1) Upewnić się, że `RESEND_API_KEY` jest ustawiony na produkcji. (2) Na produkcji brak klucza powinien być błędem (np. `500` + alert), a nie „cichym sukcesem”. (3) Najlepiej dodać kopię/powiadomienie zapasowe (np. drugi kanał), żeby żadne zgłoszenie nie ginęło.

Możliwy efekt uboczny: W trybie dev/preview (gdzie brak klucza jest celowy) nie chcemy pokazywać błędu — dlatego zmiana powinna dotyczyć tylko produkcji.

MEDIUM R2 — Brak limitu długości wiadomości

Obszar: brak walidacji rozmiaru / odporność na nadużycia

Opis: Pole „wiadomość” (i „imię i nazwisko”) mają tylko minimum (musi nie być puste), ale **nie mają maksimum**. Ktoś może wysłać wiadomość na megabajty tekstu.

Dlaczego to ryzyko: Usługa pocztowa może odrzucić wielką wiadomość (gość zobaczy błąd), a ktoś złośliwy może tym obciążać system. To też furtka do spamu „na ilość”.

Po ludzku: formularz przyjmuje „książkę telefoniczną” wklejoną w jedno pole — niepotrzebnie.

Gdzie szukać: `src/lib/contact-schema.ts:9, 16 ( name i message bez .max( ... ) )`

Rekomendacja: Dodać rozsądne limity, np. `name.max(120)` , `message.max(5000)` . Schemat jest wspólny dla przeglądarki i serwera, więc poprawka działa po obu stronach.

Możliwy efekt uboczny: Bardzo długie (i tak rzadkie) wiadomości zostaną odrzucone z komunikatem — limit warto dobrać hojnie.

MEDIUM R3 — Limit zapytań żyje tylko w jednej instancji i znika po „uśpieniu”

Obszar: stan trzymany w pamięci / brak wspólnego licznika

Opis: Licznik „3 na 10 minut” trzymany jest w pamięci pojedynczej instancji funkcji. Vercel uruchamia wiele instancji równolegle i usypia je — więc licznik bywa zerowany i nie jest współdzielony.

Dlaczego to ryzyko: Ochrona jest „dziurawa”: spamer trafiający na różne instancje (lub po uśpieniu) dostaje świeży limit. To osłabia zabezpieczenie, na którym polegasz (zwłaszcza razem z S3 i S4).

Po ludzku: kilku ochroniarzy, z których każdy liczy gości osobno i co chwilę zapomina — łatwo przejść kilka razy.

Gdzie szukać: `src/lib/rate-limit.ts:1–11` (komentarz wprost to przyznaje) · używane w `src/app/api/kontakt/route.ts:65–66`

Rekomendacja: Do prawdziwego limitu potrzebny wspólny magazyn (np. Upstash Redis z Vercel Marketplace). Patrz „Decyzje architektoniczne”. Na dziś główną ochroną pozostaje Turnstile (S4).

Możliwy efekt uboczny: Dodanie zewnętrznego magazynu to nowa zależność (może paść). Trzeba zdecydować: gdy magazyn nie odpowiada — przepuszczać czy blokować (patrz wzajemna recenzja).

MEDIUM R4 — Jeden błędny plik treści może wywalić całą sekcję projektów

Obszar: obsługa błędów / odporność na złe dane

Opis: Wczytywanie projektów i galerii „rzuca wyjątkiem” (przerywa działanie), gdy w pliku brakuje wymaganego pola albo kategoria jest spoza listy. Ponieważ ta funkcja zasila wiele miejsc, jeden zepsuty wpis psuje całą listę.

Dlaczego to ryzyko: Przy budowaniu strony — build się wywali (nie opublikujesz). Na trasie dynamicznej — gość zobaczy błąd zamiast listy projektów. Wystarczy jedna pomyłka w CMS.

Po ludzku: jedno krzywo wpięte zdjęcie w albumie sprawia, że nie otwiera się cały album, a nie tylko ta jedna strona.

Gdzie szukać: `src/lib/content.ts:99–117` (projekty) · `:283–288` (galerie)

Rekomendacja: Rozważyć „pomiń i zaloguj ostrzeżenie” dla pojedynczego błędnego pliku zamiast wywracania całości — albo przynajmniej czytelny komunikat z nazwą pliku (to akurat już jest) i świadomy wybór, czy build ma padać. Keystatic w dużej mierze pilnuje pól, więc ryzyko realne głównie przy ręcznej edycji.

Możliwy efekt uboczny: „Pomijanie” błędnych wpisów może ukryć fakt, że projekt zniknął ze strony — dlatego konieczne logowanie ostrzeżeń.

MEDIUM R6 — Słaby wgląd w awarie: gdy maile padają, nikt o tym nie wie

Obszar: obserwowalność / brak alertów

Opis: Gdy wysyłka maila się nie powiedzie, błąd trafia tylko do logów (`console.error`). Nikt ich nie ogląda na bieżąco.

Dlaczego to ryzyko: Razem z R1 oznacza, że awaria poczty może trwać tygodniami niezauważona — a każdy taki dzień to potencjalnie utracone zapytania.

Po ludzku: czujka dymu, która „mruga w piwnicy”, ale nie ma syreny — pożar zauważysz, dopiero gdy będzie za późno.

Gdzie szukać: `src/app/api/kontakt/route.ts:134, 138`

Rekomendacja: Dodać proste powiadomienie o błędzie (np. mail awaryjny innym kanałem, alert w Vercelu lub usługa typu Sentry). Minimum: regularnie zaglądać w logi Resend.

Możliwy efekt uboczny: Dodatkowa usługa/koszt; trzeba unikać „zalewu” powiadomieniami.

LOW R5 — Zapytanie do Turnstile bez limitu czasu (może „wisieć”)

Obszar: brak timeoutu

Opis: Sprawdzenie tokena u Cloudflare nie ma ustawionego limitu czasu. Jeśli Cloudflare odpowiada wolno, całe wysłanie formularza czeka aż do twardego limitu funkcji.

Dlaczego to ryzyko: Rzadkie, ale przy awarii Cloudflare gość długo patrzy na „Wysyłanie...”, a potem dostaje błąd.

Po ludzku: dzwonicz, żeby potwierdzić rezerwację, nikt nie odbiera, a Ty trzymasz słuchawkę w nieskończoność, zamiast odłożyć po chwili.

Gdzie szukać: `src/app/api/kontakt/route.ts:32–39` (`fetch` bez `AbortSignal.timeout`)

Rekomendacja: Dodać limit czasu (np. 5 s) i zdecydować, co przy przekroczeniu (raczej odmówić niż przepuścić).

Możliwy efekt uboczny: Przy bardzo wolnym (ale działającym) Cloudflare zgłoszenie może zostać odrzucone — limit warto dobrać z zapasem.

LOW R7 — Zapis ustawień hero bez obsługi błędów i ochrony przed równoczesnym zapisem (tylko dev)

Obszar: obsługa błędów / wyścig (race condition)

Opis: Endpoint `save-hero` czyta i parsuje plik JSON bez zabezpieczenia — jeśli plik byłby uszkodzony, poleci nieobsłużony błąd. Dwa zapisy naraz mogą sobie nadpisać zmiany. Działa **tylko lokalnie** (na produkcji zwraca 404), więc wpływ jest mały.

Dlaczego to ryzyko: Tylko podczas Twojej pracy lokalnej; w najgorszym razie utrata jednej zmiany kadru.

Po ludzku: dwie osoby edytują tę samą kartkę naraz — wygrywa ta, która zapisze później, druga zmiana przepada.

Gdzie szukać: `src/app/api/admin/save-hero/route.ts:43–69`

Rekomendacja: Opakować odczyt/parse w obsługę błędów z czytelnym komunikatem. Wyścig — pomijalny dla narzędzia jednoosobowego.

Możliwy efekt uboczny: Brak.

4. Recenzenci podważają się nawzajem

Każda poprawka może popsuć coś innego. Oto jak trzej recenzenci sprawdzili swoje propozycje.

Bezpieczeństwo ocenia poprawki Wydajności

- **P1 (zapamiętywanie listy projektów):** bezpieczne. Cache treści, która i tak jest publiczna, nie tworzy ryzyka wycieku. ✓ Uwaga: nie wolno cache'ować *odpowiedzi maila ani tokenów* — ale tego nikt nie proponuje.
- **P2 (limity kompresji):** wręcz wzmacnia bezpieczeństwo — niższy limit pliku to mniejsza powierzchnia nadużycia. Zgoda, ale to nie zastępuje zamknięcia dostępu (S2). Najpierw S2.

Wydajność ocenia poprawki Bezpieczeństwa

- **S1 (nagłówki/CSP):** koszt wydajności zerowy — nagłówki to kilka bajtów. ✓ Tylko nie ustawiać agresywnego cache na odpowiedziach API (i tak nie ma).
- **S2 (zamknięcie kompresji) i S3 (zaufany adres IP):** sprawdzenie tożsamości i odczyt innego nagłówka są błyskawiczne — brak wpływu na szybkość. ✓
- **S4 (twarde wymaganie Turnstile na produkcji):** dokłada jedno zapytanie sieciowe do Cloudflare — ale ono już istnieje, gdy klucz jest ustawiony. Bez realnej zmiany kosztu.

Niezawodność ocenia, czy poprawki nie wprowadzą nowych awarii

- **S4 + R1 (brak klucza = błąd na produkcji):** ⚠ uwaga! Trzeba to ograniczyć wyłącznie do produkcji. Inaczej dev/preview (gdzie celowo nie ma kluczy) zacznie zwracać błędy i utrudni pracę. Rozwiązanie: warunek „tylko gdy `NODE_ENV === production`”.
- **R3 (wspólny licznik w Redis):** ⚠ nowa zależność = nowe ryzyko awarii. Jeśli Redis padnie i wybierzemy „blokuj przy błędzie”, formularz przestanie działać dla wszystkich. **Rekomendacja: przy awarii magazynu przepuszczać (fail-open) i logować** — spam jest mniej groźny niż zablokowany kontakt. Bezpieczeństwo akceptuje ten kompromis, bo Turnstile zostaje drugą warstwą.
- **P1 (cache listy):** ⚠ jeśli kiedyś dojdzie odświeżanie treści „na żywo” bez przebudowy, cache mógłby pokazać stare dane. Dziś treść zmienia się przez nowe wdrożenie, więc bezpiecznie — ale warto to udokumentować, by ktoś w przyszłości nie wpadł w pułapkę „nieaktualnej listy”.
- **R4 (pomijanie błędnych plików):** ⚠ „ciche pomijanie” może ukryć zniknięcie projektu. Konieczny widoczny log/ostrzeżenie, najlepiej też w trakcie budowania.

5. Do naprawy natychmiast (CRITICAL i HIGH)

Nie wykryto żadnego problemu klasy **CRITICAL**. Poniższe trzy to **HIGH** — najpierw te:

#	Problem (po ludzku)	Gdzie	Dlaczego pilne
R1	Formularz mówi „Dziękuję”, choć mail może nie dotrzeć	kontakt/route.ts:110–116 · contact-form.tsx:92	Tracisz zapytania klientów i nawet o tym nie wiesz
S2	Otwarte narzędzie kompresji — każdy z internetu może je obciążać	api/admin/compress-image/route.ts:28–67	Koszty + ryzyko przeciążenia strony
S1	Zabezpieczenia przeglądarki nie działają na żywej stronie (są w pliku Netlify, a hosting to Vercel)	netlify.toml:18–28 · next.config.ts	Brak ochrony m.in. przed clickjackingiem

6. Do backlogu (MEDIUM i LOW)

#	Waga	Problem	Gdzie
S3	MEDIUM	Limit można obejść podrobionym adresem IP	kontakt/route.ts:13–21
S4	MEDIUM	Antyspam wyłącza się sam przy braku klucza	kontakt/route.ts:28–29
P1	MEDIUM	Lista projektów czytana wielokrotnie bez cache	content.ts:180–236
P2	MEDIUM	Kompresja trzyma duży plik w pamięci, bez limitu równoległości	compress-image/route.ts:53–61
R2	MEDIUM	Brak maksymalnej długości wiadomości	contact-schema.ts:9,16
R3	MEDIUM	Limit zapytań nie jest współdzielony i znika po uśpieniu	rate-limit.ts:1–11
R4	MEDIUM	Jeden błędny plik treści wywala całą sekcję	content.ts:99–117, 283–288
R6	MEDIUM	Brak alertu, gdy maile padają	kontakt/route.ts:134,138
S5	LOW	Strona design-system / narzędzia admin publicznie dostępne	design-system/page.tsx
P3	LOW	Pamięć limitera rośnie bez sprzątania	rate-limit.ts:11–28
R5	LOW	Zapytanie do Turnstile bez limitu czasu	kontakt/route.ts:32–39
R7	LOW	save-hero bez obsługi błędów (tylko dev)	save-hero/route.ts:43–69

7. Decyzje architektoniczne do podjęcia

To wybory dla Ciebie (lub Twojego programisty) — nie „naprawa”, tylko kierunek.

1. **Jeden hosting, nie dwa.** W repo są ślady i Netlify (`netlify.toml`), i Vercela (`.vercel/`). Żywa strona stoi na Vercelu. Decyzja: oficjalnie wybrać Vercela i przenieść tam wszystkie ustawienia (zwłaszcza nagłówki

bezpieczeństwa — S1), albo świadomie wrócić na Netlify. Trzymanie obu po cichu = ustawienia „znikają”.

2. **Limit zapytań: pamięć czy wspólny magazyn?** Dziś „w pamięci” (zawodny, R3). Opcje: zostawić jak jest i polegać głównie na Turnstile, albo dołożyć wspólny magazyn (np. Upstash Redis z Vercel Marketplace). Jeśli magazyn — z góry ustalić: przy jego awarii *przepuszczać* (rekomendacja) czy *blokować*.

3. **Narzędzia /admin na produkcji.** Zdecydować, czy `/admin/compress` ma w ogóle istnieć online. Opcje: tylko lokalnie (jak `save-hero`), albo za logowaniem. Dziś jest otwarte (S2).

4. **Czy formularz ma gwarancję dostarczenia?** Rozważyć drugi, niezależny kanał zapisu zgłoszeń (np. kopia do innej skrzynki / prosty zapis), żeby awaria jednej usługi nie gubiła klientów (powiązane z R1).

8. Testy, które warto dopisać

„Test” to mały programik, który automatycznie sprawdza, że dana rzecz działa — żeby przyszła zmiana czegoś nie zepsuła po cichu.

Formularz kontaktowy — endpoint `/api/kontakt`

- Niepoprawny JSON w żądaniu → odpowiedź 400 (a nie awaria).
- Wypełniona pułapka (pole „firma”) → udawany sukces, mail NIE wychodzi.
- Czwarte zgłoszenie w 10 minut → odpowiedź 429 (limit) z nagłówkiem `Retry-After`.
- Błędny token Turnstile → 400; brak klucza na produkcji → błąd (po poprawce S4/R1), w dev → przepuszczenie.
- Resend zwraca błąd → 502 i komunikat zastępczy.
- **Kluczowe (R1):** brak `RESEND_API_KEY` na produkcji → użytkownik NIE widzi fałszywego „Dziękuję”.
- Wiadomość ponad limit długości (po poprawce R2) → 400.

Walidacja danych — `contact-schema.ts`

- Poprawny i niepoprawny e-mail; brak zgody RODO → błąd; nowe limity długości.

Odczyt adresu IP — `getClientIp`

- Próba podrobienia `x-forwarded-for` → po poprawce S3 brany jest adres zaufany.

Wczytywanie treści — `content.ts`

- Plik z brakującym polem / złą kategorią → czytelny błąd z nazwą pliku (lub pominięcie z ostrzeżeniem, zależnie od decyzji R4).

Kompresja — `/api/admin/compress-image`

- Plik niebędący obrazem → 415; plik za duży → 413; po poprawce S2: brak dostępu bez uprawnień.

9. Krótkie podsumowanie dla właściciela produktu

Twoja strona jest napisana solidnie i schludnie — widać dbałość o szczegóły (walidacja formularza po obu stronach, pułapka na boty, sensowne komentarze). **Nie ma niczego, co „się pali”** (zero problemów klasy CRITICAL).

Są jednak **trzy rzeczy do pilnej naprawy**:

1. **Najważniejsza**: w pewnych warunkach klient wyśle wiadomość, zobaczy „Dziękuję”, a **mail do Ciebie nie dotrze** — i nikt się nie zorientuje. Najpierw upewnij się, że klucz do wysyłki maili (`RESEND_API_KEY`) jest wpisany na Vercelu, a docelowo niech brak tego klucza zgłasza błąd zamiast udawać sukces. (R1)
2. Masz **otwarte „ksero na ulicy”** — narzędzie do kompresji obrazów, którego może użyć każdy z internetu i naliczyć Ci koszty albo zamulić stronę. Zamknąć je albo schować za logowaniem. (S2)
3. Twoje **zabezpieczenia przeglądarki są wpięte w plik, którego Twój hosting nie czyta** — czyli realnie nie działają. Przenieść je tam, gdzie zadziałają (Vercel). (S1)

Reszta (12 punktów) to sprawy „średnie i drobne” do backlogu — głównie wzmocnienie ochrony przed spamem i porządki pod większy ruch. Nie blokują uruchomienia strony.

Co zrobić w tym tygodniu: (1) sprawdzić klucze na Vercelu (mail + antyspam), (2) zlecić zamknięcie narzędzia kompresji, (3) przenieść nagłówki bezpieczeństwa. To trzy konkretne zadania — po nich strona jest gotowa „na produkcję” z czystym sumieniem.

Słowniczek (proste tłumaczenia)

Endpoint

Adres w aplikacji, pod który przeglądarka „wysyła paczkę” (np. `/api/kontakt` przyjmuje treść formularza).

Nagłówki bezpieczeństwa / CSP

Krótkie instrukcje, które serwer wysyła przeglądarce: „nie pozwalaj na te niebezpieczne rzeczy”. Bez nich przeglądarka jest mniej ostrożna.

Clickjacking

Podstęp: ktoś osadza Twoją stronę w niewidzialnej ramce na swojej i „kradnie” kliknięcia użytkownika.

Rate limiting (limit zapytań)

Ograniczenie „ile razy w danym czasie ktoś może coś wysłać” — żeby spamer nie zalał formularza.

Honeypot (pułapka)

Ukryte pole, którego człowiek nie widzi. Jeśli zostanie wypełnione, wiadomo że to bot.

Turnstile

Cloudflare’owy „test, czy jesteś człowiekiem” (nowsza wersja CAPTCHA).

Cache (zapamiętywanie)

Trzymanie gotowego wyniku, żeby nie liczyć go w kółko od nowa.

Cold start (zimny start) / instancja

Serwer „usypia”, gdy nikt nie korzysta, i „budzi się” przy nowym ruchu — wtedy traci to, co miał w pamięci.

Race condition (wyścig)

Dwie operacje dzieją się naraz i wchodzi sobie w drogę — wynik bywa przypadkowy.

Fail-open / fail-closed

Co zrobić, gdy zabezpieczenie samo się zepsuje: „fail-open” = przepuść (wygoda), „fail-closed” = zablokuj (bezpieczeństwo).

DoS (odmowa usługi)

Zasypanie serwera tak dużą liczbą żądań, że przestaje odpowiadać zwykłym gościom.

Raport wygenerowany na podstawie realnego kodu repozytorium (przeczytane pliki: konfiguracje, 5 endpointów API, formularz, biblioteki treści/limitu/walidacji, panele admin, middleware, layout). Dokument prywatny — zapisany w `docs/`, celowo poza `public/`, więc nie jest publikowany na stronie. · Michał Stężyła — portfolio · 11.06.2026